

```
    ],  
    "ps": ["machine-a.example.com:2221"] # /job:ps/task:0  
}
```

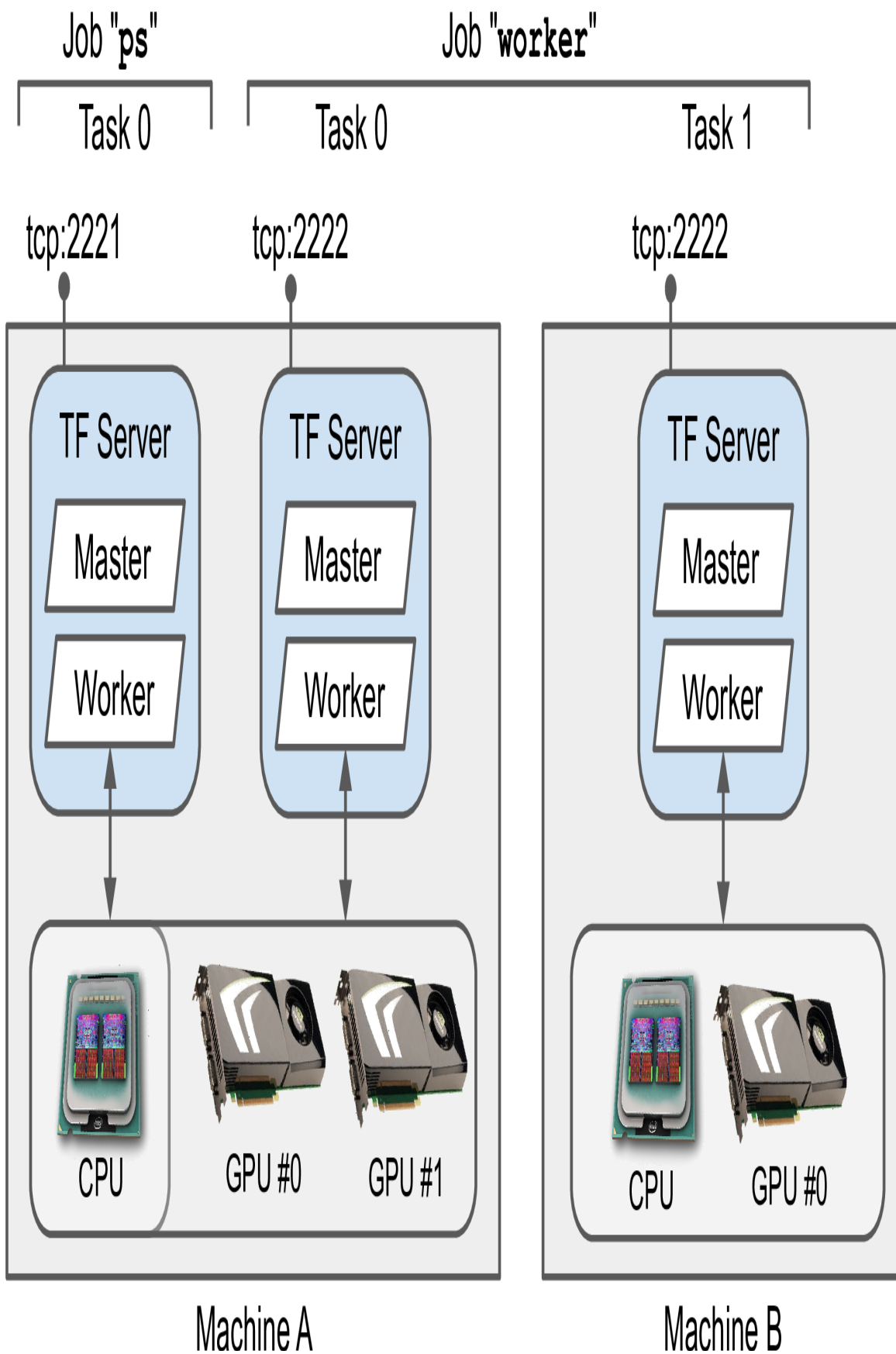


Figure 19-18. TensorFlow cluster

In general there will be a single task per machine, but as this example shows, you can configure multiple tasks on the same machine if you want. In this case, if they share the same GPUs, make sure the RAM is split appropriately, as discussed earlier.

### WARNING

By default, every task in the cluster may communicate with every other task, so make sure to configure your firewall to authorize all communications between these machines on these ports (it's usually simpler if you use the same port on every machine).

When you start a task, you must give it the cluster spec, and you must also tell it what its type and index are (e.g., worker #0). The simplest way to specify everything at once (both the cluster spec and the current task's type and index) is to set the `TF_CONFIG` environment variable before starting TensorFlow. It must be a JSON-encoded dictionary containing a cluster specification (under the `"cluster"` key) and the type and index of the current task (under the `"task"` key). For example, the following `TF_CONFIG` environment variable uses the cluster we just defined and specifies that the task to start is the worker #0:

```
os.environ["TF_CONFIG"] = json.dumps({
    "cluster": cluster_spec,
    "task": {"type": "worker", "index": 0}
})
```

### TIP

In general you want to define the `TF_CONFIG` environment variable outside of Python, so the code does not need to include the current task's type and index (this makes it possible to use the same code across all workers).

Now let's train a model on a cluster! We will start with the mirrored strategy. First, you need to set the `TF_CONFIG` environment variable appropriately for each task. There should be no parameter server (remove the "ps" key in the cluster spec), and in general you will want a single worker per machine. Make extra sure you set a different task index for each task. Finally, run the following script on every worker:

```
import tempfile
import tensorflow as tf

strategy = tf.distribute.MultiWorkerMirroredStrategy() # at the start!
resolver =
tf.distribute.cluster_resolver.TFConfigClusterResolver()
print(f"Starting task {resolver.task_type} #{resolver.task_id}")
[...] # Load and split the MNIST dataset

with strategy.scope():
    model = tf.keras.Sequential([...]) # build the Keras model
    model.compile([...]) # compile the model

model.fit(X_train, y_train, validation_data=(X_valid, y_valid),
epochs=10)

if resolver.task_id == 0: # the chief saves the model to the right location
    model.save("my_mnist_multiworker_model", save_format="tf")
else:
    tmpdir = tempfile.mkdtemp() # other workers save to a temporary directory
    model.save(tmpdir, save_format="tf")
    tf.io.gfile.rmtree(tmpdir) # and we can delete this directory at the end!
```

That's almost the same code we used earlier, except this time we are using the `MultiWorkerMirroredStrategy`. When you start this script on the first workers, they will remain blocked at the AllReduce step, but training will begin as soon as the last worker starts up, and you will see them all advancing at exactly the same rate, since they synchronize at each step.

## WARNING

When using the `MultiWorkerMirroredStrategy`, it's important to ensure that all workers do the same thing, including saving model checkpoints, or writing TensorBoard logs, even though we will only keep what the chief writes. This is because these operations may need to run the AllReduce operations, so all workers must be in sync.

There are two AllReduce implementations for this distribution strategy: a ring AllReduce algorithm based on gRPC for the network communications, and NCCL's implementation. The best algorithm to use depends on the number of workers, the number and types of GPUs, and the network. By default, TensorFlow will apply some heuristics to select the right algorithm for you, but you can force NCCL (or RING) like this:

```
strategy = tf.distribute.MultiWorkerMirroredStrategy(  
  
communication_options=tf.distribute.experimental.CommunicationOptions(  
  
implementation=tf.distribute.experimental.CollectiveCommunication.NCCL))
```

If you prefer to implement asynchronous data parallelism with parameter servers, change the strategy to `ParameterServerStrategy`, add one or more parameter servers, and configure `TF_CONFIG` appropriately for each task. Note that although the workers will work asynchronously, the replicas on each worker will work synchronously.

Lastly, if you have access to **TPUs on Google Cloud**—for example if you use Colab and you set the accelerator type to TPU—then you can create a `TPUStrategy` like this:

```
resolver = tf.distribute.cluster_resolver.TPUClusterResolver()  
tf.tpu.experimental.initialize_tpu_system(resolver)  
strategy = tf.distribute.experimental.TPUStrategy(resolver)
```

This needs to be run right after importing TensorFlow. You can then use this strategy normally.

### TIP

If you are a researcher, you may be eligible to use TPUs for free; see <https://tensorflow.org/tfrc> for more details.

You can now train models across multiple GPUs and multiple servers: give yourself a pat on the back! If you want to train a very large model, you will need many GPUs, across many servers, which will require either buying a lot of hardware or managing a lot of cloud virtual machines. In many cases, it's less hassle and less expensive to use a cloud service that takes care of provisioning and managing all this infrastructure for you, just when you need it. Let's see how to do that using Vertex AI.

## Running Large Training Jobs on Vertex AI

Vertex AI allows you to create custom training jobs with your own training code. In fact, you can use almost the same training code as you would use on your own TF cluster. The main thing you must change is where the chief should save the model, the checkpoints, and the TensorBoard logs. Instead of saving the model to a local directory, the chief must save it to GCS, using the path provided by Vertex AI in the `AIP_MODEL_DIR` environment variable. For the model checkpoints and TensorBoard logs, you should use the paths contained in the `AIP_CHECKPOINT_DIR` and `AIP_TENSORBOARD_LOG_DIR` environment variables, respectively. Of course, you must also make sure that the training data can be accessed from the virtual machines, such as on GCS, or another GCP service like BigQuery, or directly from the Web. Lastly, Vertex AI sets the "chief" task type explicitly, so you should identify the chief using `resolved.task_type == "chief"` instead of `resolved.task_id == 0`.

```

import os
[...] # other imports, create MultiWorkerMirroredStrategy, and
resolver

if resolver.task_type == "chief":
    model_dir = os.getenv("AIP_MODEL_DIR") # paths provided by
Vertex AI
    tensorboard_log_dir = os.getenv("AIP_TENSORBOARD_LOG_DIR")
    checkpoint_dir = os.getenv("AIP_CHECKPOINT_DIR")
else:
    tmp_dir = Path(tempfile.mkdtemp()) # other workers use a
temporary dirs
    model_dir = tmp_dir / "model"
    tensorboard_log_dir = tmp_dir / "logs"
    checkpoint_dir = tmp_dir / "ckpt"

callbacks = [tf.keras.callbacks.TensorBoard(tensorboard_log_dir),
              tf.keras.callbacks.ModelCheckpoint(checkpoint_dir)]
[...] # build and compile using the strategy scope, just like
earlier
model.fit(X_train, y_train, validation_data=(X_valid, y_valid),
epochs=10,
          callbacks=callbacks)
model.save(model_dir, save_format="tf")

```

### TIP

If you place the training data on GCS, you can create a `tf.data.TextLineDataset` or `tf.data.TFRecordDataset` to access it: just use the GCS paths as the filenames (e.g., `gs://my_bucket/data/001.csv`). These datasets rely on the `tf.io.gfile` package to access files: it supports both local files and GCS files.

Now we can create a custom training job on Vertex AI, based on this script. We specify the job name, the path to our training script, the Docker image to use for training, the one to use for predictions (after training), any additional Python libraries you may need, and lastly the bucket that Vertex AI should use as a staging directory to store the training script. By default, that's also where the training script will save the trained model, as well as the TensorBoard logs and model checkpoints (if any). Let's create the job:

```

custom_training_job = aiplatform.CustomTrainingJob(
    display_name="my_custom_training_job",
    script_path="my_vertex_ai_training_task.py",
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-
4:latest",
    model_serving_container_image_uri=server_image,
    requirements=["gcsfs==2022.3.0"], # not needed, this is just
an example
    staging_bucket=f"gs://{bucket_name}/staging"
)

```

And now let's run it on two workers, each with 2 GPUs:

```

mnist_model2 = custom_training_job.run(
    machine_type="n1-standard-4",
    replica_count=2,
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2,
)

```

And that's it: Vertex AI will provision the compute nodes you requested (within your quotas), and it will run your training script across them. Once the job is complete, the `run()` method will return a trained model that you can use exactly like the one you created earlier: you can deploy it to an endpoint, or use it to make batch predictions. If anything goes wrong during training, you can view the logs in the GCP console: in the ☰ navigation menu, select *Vertex AI* → *Training*, click on your training job, and click on *VIEW LOGS*. Alternatively, you can click on the *CUSTOM JOBS* tab and copy the job's ID (e.g., 1234), then select *Logging* from the ☰ navigation menu, and query `resource.labels.job_id=1234`.

### TIP

To visualize the training progress, just start TensorBoard and point its `--logdir` to the GCS path of the logs. It will use *application default credentials*, which you can setup using `gcloud auth application-default login`. Vertex AI also offers hosted TensorBoard servers if you prefer.



If you want to try out a few hyperparameter values, one option is to run multiple jobs. You can pass the hyperparameter values to your script as command line arguments by setting the `args` parameter when calling the `run()` method. Or you can pass them as environment variables using the `environment_variables` parameter.

However, if you want to run a large hyperparameter tuning job on the cloud, a much better option is to use Vertex AI's hyperparameter tuning service. Let's see how.

## Hyperparameter Tuning on Vertex AI

Vertex AI's hyperparameter tuning service is based on a Bayesian optimization algorithm, capable of quickly finding optimal combinations of hyperparameters: to use it, you first need to create a training script that accepts hyperparameter values as command line arguments. For example, your script could use the `argparse` standard library like this:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--n_hidden", type=int, default=2)
parser.add_argument("--n_neurons", type=int, default=256)
parser.add_argument("--learning_rate", type=float, default=1e-2)
parser.add_argument("--optimizer", default="adam")
args = parser.parse_args()
```

The hyperparameter tuning service will call your script multiple times, each time with different hyperparameter values: each run is called a *trial*, and the set of trials is called a *study*. Your training script must then use the given hyperparameter values to build and compile a model. You can use a mirrored distribution strategy if you want, in case each trial runs on a multi-GPU machine. Then the script can load the dataset and train the model. For example:

```
import tensorflow as tf

def build_model(args):
```

```

    with tf.distribute.MirroredStrategy().scope():
        model = tf.keras.Sequential()
        model.add(tf.keras.layers.Flatten(input_shape=[28, 28],
dtype=tf.uint8))
        for _ in range(args.n_hidden):
            model.add(tf.keras.layers.Dense(args.n_neurons,
activation="relu"))
        model.add(tf.keras.layers.Dense(10,
activation="softmax"))
        opt = tf.keras.optimizers.get(args.optimizer)
        opt.learning_rate = args.learning_rate
        model.compile(loss="sparse_categorical_crossentropy",
optimizer=opt,
                    metrics=["accuracy"])
        return model

[...]
```

*# load the dataset*  
model = build\_model(args)  
history = model.fit([...])

### TIP

You can use the AIP\_\* environment variables we discussed earlier to know where to save the checkpoints, the TensorBoard logs, and the final model.

Lastly, the script must report the model's performance back to Vertex AI's hyperparameter tuning service, so it can decide which hyperparameters to try next. For this, you must use the `hyper tune` library, which is automatically installed on Vertex AI training VMs:

```

import hypertune

hypertune = hypertune.HyperTune()
hypertune.report_hyperparameter_tuning_metric(
    hyperparameter_metric_tag="accuracy", # name of the reported
metric
    metric_value=max(history.history["val_accuracy"]), # metric
value
    global_step=model.optimizer.iterations.numpy(),
)
```

Now that your training script is ready, you need to define the type of machine you would like to run it on. For this, you must define a custom job, which Vertex AI will use as a template for each trial:

```
trial_job = aiplatform.CustomJob.from_local_script(
    display_name="my_search_trial_job",
    script_path="my_vertex_ai_trial.py", # path to your training
script
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-
4:latest",
    staging_bucket=f"gs://{bucket_name}/staging",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2, # in this example, each trial will have
2 GPUs
)
```

Finally, you're ready to create and run the hyperparameter tuning job:

```
from google.cloud.aiplatform import hyperparameter_tuning as hpt

hp_job = aiplatform.HyperparameterTuningJob(
    display_name="my_hp_search_job",
    custom_job=trial_job,
    metric_spec={"accuracy": "maximize"},
    parameter_spec={
        "learning_rate": hpt.DoubleParameterSpec(min=1e-3,
max=10, scale="log"),
        "n_neurons": hpt.IntegerParameterSpec(min=1, max=300,
scale="linear"),
        "n_hidden": hpt.IntegerParameterSpec(min=1, max=10,
scale="linear"),
        "optimizer": hpt.CategoricalParameterSpec(["sgd",
"adam"]),
    },
    max_trial_count=100,
    parallel_trial_count=20,
)
hp_job.run()
```

Here, we tell Vertex AI to maximize the metric named "accuracy": this name must match the name of the metric reported by the training script. We also define the search space, using a log scale for the learning rate, and a linear scale (i.e., uniform) for the other hyperparameters. The

hyperparameter names must match the command line arguments of the training script. Lastly, we set the maximum number of trials to 100, and the maximum number of trials running in parallel to 20. If you increase the number of parallel trials to (say) 60, then the total search time will be reduced significantly, by a factor of up to 3. However, the first 60 trials will be started in parallel, so they will not benefit from the other trials' feedback. Therefore, you should increase the max number of trials to compensate, for example up to about 140.

This will take quite a while. Once the job is completed, you can fetch the trial results using `hp_job.trials`. Each trial result is represented as a protobuf object, containing the hyperparameter values and the resulting metrics. Let's find the best trial:

```
def get_final_metric(trial, metric_id):
    for metric in trial.final_measurement.metrics:
        if metric.metric_id == metric_id:
            return metric.value

trials = hp_job.trials
trial accuracies = [get_final_metric(trial, "accuracy") for trial
in trials]
best_trial = trials[np.argmax(trial accuracies)]
```

Now let's look at this trial's accuracy, and its hyperparameter values:

```
>>> max(trial accuracies)
0.977400004863739
>>> best_trial.id
'98'
>>> best_trial.parameters
[parameter_id: "learning_rate" value { number_value: 0.001 },
 parameter_id: "n_hidden" value { number_value: 8.0 },
 parameter_id: "n_neurons" value { number_value: 216.0 },
 parameter_id: "optimizer" value { string_value: "adam" }
]
```

That's it! Now you can get this trial's saved model, optionally train it a bit more, and deploy it to production.

## TIP

Vertex AI also includes an AutoML service which completely takes care of finding the right model architecture and training it for you. All you need to do is upload your dataset to Vertex AI using a special format that depends on the type of dataset (images, text, tabular, video...), then create an AutoML training job, pointing to the dataset and specifying the maximum number of compute hours you're willing to spend. See the notebook for an example.

## HYPERPARAMETER TUNING USING KERAS TUNER ON VERTEX AI

Instead of using Vertex AI's hyperparameter tuning service, you can use Keras Tuner (introduced in [Chapter 10](#)) and run it on Vertex AI VMs. Keras Tuner provides a simple way to scale hyperparameter search by distributing it across multiple machines: it only requires setting three environment variables on each machine, then running your regular Keras Tuner code on each machine. You can use the exact same script on all machines. One of the machines acts as the chief (i.e., the oracle), and the others act as workers. Each worker asks the chief which hyperparameter values to try, then the worker trains the model using these hyperparameter values, and finally it reports the model's performance back to the chief, which can then decide which hyperparameter values the worker should try next.

The three environment variables you need to set on each machine are:

- `KERASTUNER_TUNER_ID`: equal to "chief" on the chief machine, or a unique identifier on each worker machine, such as "worker0", "worker1", etc.
- `KERASTUNER_ORACLE_IP`: the IP address or hostname of the chief machine. The chief itself should generally use "0.0.0.0" to listen on every IP address on the machine.
- `KERASTUNER_ORACLE_PORT`: the TCP port that the chief will be listening on.

You can use distributed Keras Tuner on any set of machines. If you want to run it on Vertex AI machines, then you can spawn a regular training job, and just modify the training script to set the environment variables properly before using Keras Tuner. See the notebook for an example.

Now you have all the tools and knowledge you need to create state-of-the-art neural net architectures and train them at scale using various distribution strategies, on your own infrastructure or on the cloud, and then deploy them anywhere. In other words, you now have superpowers: use them well!

## Exercises

1. What does a SavedModel contain? How do you inspect its content?
2. When should you use TF Serving? What are its main features? What are some tools you can use to deploy it?
3. How do you deploy a model across multiple TF Serving instances?
4. When should you use the gRPC API rather than the REST API to query a model served by TF Serving?
5. What are the different ways TFLite reduces a model's size to make it run on a mobile or embedded device?
6. What is quantization-aware training, and why would you need it?
7. What are model parallelism and data parallelism? Why is the latter generally recommended?
8. When training a model across multiple servers, what distribution strategies can you use? How do you choose which one to use?
9. Train a model (any model you like) and deploy it to TF Serving or Google Vertex AI. Write the client code to query it using the REST API or the gRPC API. Update the model and deploy the new version. Your client code will now query the new version. Roll back to the first version.
10. Train any model across multiple GPUs on the same machine using the `MirroredStrategy` (if you do not have access to GPUs, you can use Colaboratory with a GPU Runtime and create two logical GPUs).

Train the model again using the `CentralStorageStrategy` and compare the training time.

11. Fine-tune a model of your choice on Vertex AI, using either Keras Tuner or Vertex AI's hyperparameter tuning service.

## Thank You!

Before we close the last chapter of this book, I would like to thank you for reading it up to the last paragraph. I truly hope that you had as much pleasure reading this book as I had writing it, and that it will be useful for your projects, big or small.

If you find errors, please send feedback. More generally, I would love to know what you think, so please don't hesitate to contact me via O'Reilly, through the *ageron/handson-ml3* GitHub project, or on Twitter at [@aureliengeron](https://twitter.com/aureliengeron).

Going forward, my best advice to you is to practice and practice: try going through all the exercises (if you have not done so already), play with the notebooks, join Kaggle.com or some other ML community, watch ML courses, read papers, attend conferences, and meet experts. Things move fast so try to keep up to date: several YouTube channels regularly present Deep Learning papers in great detail, in a very approachable way. I particularly recommend channels by Yannic Kilcher, Letitia Parcalabescu, and Xander Steenbrugge. For fascinating ML discussions and higher level insights, make sure to check out ML Street Talk, and Lex Fridman's channel. It also helps tremendously to have a concrete project to work on, whether it is for work or for fun (ideally for both), so if there's anything you have always dreamt of building, give it a shot! Work incrementally; don't shoot for the moon right away, but stay focused on your project and build it piece by piece. It will require patience and perseverance, but when you have a walking robot, or a working chatbot, or whatever else you fancy to build, it will be immensely rewarding.



My greatest hope is that this book will inspire you to build a wonderful ML application that will benefit all of us! What will it be?  
—*Aurélien Géron, April, 2022*

---

- 1 An A/B experiment consists in testing two different versions of your product on different subsets of users in order to check which version works best and get other insights.
- 2 Google AI Platform (formerly known as Google ML Engine) and Google AutoML merged in 2021 to form Google Vertex AI.
- 3 A REST (or RESTful) API is an API that uses standard HTTP verbs, such as GET, POST, PUT, and DELETE, and uses JSON inputs and outputs. The gRPC protocol is more complex but more efficient. Data is exchanged using protocol buffers (see [Chapter 13](#)).
- 4 If you are not familiar with Docker, it allows you to easily download a set of applications packaged in a *Docker image* (including all their dependencies and usually some good default configuration) and then run them on your system using a *Docker engine*. When you run an image, the engine creates a *Docker container* that keeps the applications well isolated from your own system—but you can give it some limited access if you want. It is similar to a virtual machine, but much faster and lighter, as the container relies directly on the host’s kernel. This means that the image does not need to include or run its own kernel.
- 5 There are also GPU images available, and other installation options. For more details, please check out the official [installation instructions](#).
- 6 To be fair, this can be mitigated by serializing the data first and encoding it to Base64 before creating the REST request. Moreover, REST requests can be compressed using gzip, which reduces the payload size significantly.
- 7 Also check out TensorFlow’s [Graph Transform Tool](#) for modifying and optimizing computational graphs.
- 8 For example, a PWA must include icons of various sizes for different mobile devices, it must be served via HTTPS, it must include a manifest file containing metadata such as the name of the app and the background color.
- 9 Please check the TensorFlow docs for detailed and up-to-date installation instructions, as they change quite often.
- 10 As we saw in [Chapter 12](#), a kernel is an operation’s implementation for a specific data type and device type. For example, there is a GPU kernel for the `float32 tf.matmul()` operation, but there is no GPU kernel for `int32 tf.matmul()`, only a CPU kernel.
- 11 You can also use `tf.debugging.set_log_device_placement(True)` to log all device placements.
- 12 This can be useful if you want to guarantee perfect reproducibility, as I explain in [this video](#), based on TF 1.

- 13 At the time of this writing it only prefetches the data to the CPU RAM, but you can use `tf.data.experimental.prefetch_to_device()` to make it prefetch the data and push it to the device of your choice so that the GPU does not waste time waiting for the data to be transferred.
- 14 If the two CNNs are identical, then it is called a *Siamese neural network*.
- 15 If you are interested in going further with model parallelism, check out [Mesh TensorFlow](#).
- 16 This name is slightly confusing because it sounds like some replicas are special, doing nothing. In reality, all replicas are equivalent: they all work hard to be among the fastest at each training step, and the losers vary at every step (unless some devices are really slower than others). However, it does mean that if one or two servers crash, training will continue just fine.
- 17 Jianmin Chen et al., “Revisiting Distributed Synchronous SGD,” arXiv preprint arXiv:1604.00981 (2016).
- 18 Aaron Harlap et al., “PipeDream: Fast and Efficient Pipeline Parallel DNN Training” (2018).
- 19 Paul Barham et al., “Pathways: Asynchronous Distributed Dataflow for ML” (2022).
- 20 For more details on AllReduce algorithms, read this [great post](#) by Yuichiro Ueno, and this page on [scaling with NCCL](#).

[OceanofPDF.com](https://oceanofpdf.com)

# Appendix A. Machine Learning Project Checklist

---

This checklist can guide you through your Machine Learning projects. There are eight main steps:

1. Frame the problem and look at the big picture.
2. Get the data.
3. Explore the data to gain insights.
4. Prepare the data to better expose the underlying data patterns to Machine Learning algorithms.
5. Explore many different models and shortlist the best ones.
6. Fine-tune your models and combine them into a great solution.
7. Present your solution.
8. Launch, monitor, and maintain your system.

Obviously, you should feel free to adapt this checklist to your needs.

## Frame the Problem and Look at the Big Picture

1. Define the objective in business terms.
2. How will your solution be used?
3. What are the current solutions/workarounds (if any)?

4. How should you frame this problem (supervised/unsupervised, online/offline, etc.)?
5. How should performance be measured?
6. Is the performance measure aligned with the business objective?
7. What would be the minimum performance needed to reach the business objective?
8. What are comparable problems? Can you reuse experience or tools?
9. Is human expertise available?
10. How would you solve the problem manually?
11. List the assumptions you (or others) have made so far.
12. Verify assumptions if possible.

## **Get the Data**

Note: automate as much as possible so you can easily get fresh data.

1. List the data you need and how much you need.
2. Find and document where you can get that data.
3. Check how much space it will take.
4. Check legal obligations, and get authorization if necessary.
5. Get access authorizations.
6. Create a workspace (with enough storage space).
7. Get the data.
8. Convert the data to a format you can easily manipulate (without changing the data itself).

9. Ensure sensitive information is deleted or protected (e.g., anonymized).
10. Check the size and type of data (time series, sample, geographical, etc.).
11. Sample a test set, put it aside, and never look at it (no data snooping!).

## Explore the Data

Note: try to get insights from a field expert for these steps.

1. Create a copy of the data for exploration (sampling it down to a manageable size if necessary).
2. Create a Jupyter notebook to keep a record of your data exploration.
3. Study each attribute and its characteristics:
  - Name
  - Type (categorical, int/float, bounded/unbounded, text, structured, etc.)
  - % of missing values
  - Noisiness and type of noise (stochastic, outliers, rounding errors, etc.)
  - Usefulness for the task
  - Type of distribution (Gaussian, uniform, logarithmic, etc.)
4. For supervised learning tasks, identify the target attribute(s).
5. Visualize the data.
6. Study the correlations between attributes.
7. Study how you would solve the problem manually.

8. Identify the promising transformations you may want to apply.
9. Identify extra data that would be useful (go back to “**Get the Data**”).
10. Document what you have learned.

## Prepare the Data

Notes:

- Work on copies of the data (keep the original dataset intact).
- Write functions for all data transformations you apply, for five reasons:
  - So you can easily prepare the data the next time you get a fresh dataset
  - So you can apply these transformations in future projects
  - To clean and prepare the test set
  - To clean and prepare new data instances once your solution is live
  - To make it easy to treat your preparation choices as hyperparameters

1. Data cleaning:

- Fix or remove outliers (optional).
- Fill in missing values (e.g., with zero, mean, median...) or drop their rows (or columns).

2. Feature selection (optional):

- Drop the attributes that provide no useful information for the task.

3. Feature engineering, where appropriate:

- Discretize continuous features.

- Decompose features (e.g., categorical, date/time, etc.).
- Add promising transformations of features (e.g.,  $\log(x)$ ,  $\sqrt{x}$ ,  $x^2$ , etc.).
- Aggregate features into promising new features.

#### 4. Feature scaling:

- Standardize or normalize features.

## Shortlist Promising Models

Notes:

- If the data is huge, you may want to sample smaller training sets so you can train many different models in a reasonable time (be aware that this penalizes complex models such as large neural nets or Random Forests).
  - Once again, try to automate these steps as much as possible.
1. Train many quick-and-dirty models from different categories (e.g., linear, naive Bayes, SVM, Random Forest, neural net, etc.) using standard parameters.
  2. Measure and compare their performance.
    - For each model, use  $N$ -fold cross-validation and compute the mean and standard deviation of the performance measure on the  $N$  folds.
  3. Analyze the most significant variables for each algorithm.
  4. Analyze the types of errors the models make.
    - What data would a human have used to avoid these errors?
  5. Perform a quick round of feature selection and engineering.

6. Perform one or two more quick iterations of the five previous steps.
7. Shortlist the top three to five most promising models, preferring models that make different types of errors.

## Fine-Tune the System

Notes:

- You will want to use as much data as possible for this step, especially as you move toward the end of fine-tuning.
- As always, automate what you can.

1. Fine-tune the hyperparameters using cross-validation:

- Treat your data transformation choices as hyperparameters, especially when you are not sure about them (e.g., if you're not sure whether to replace missing values with zeros or with the median value, or to just drop the rows).
- Unless there are very few hyperparameter values to explore, prefer random search over grid search. If training is very long, you may prefer a Bayesian optimization approach (e.g., using Gaussian process priors, [as described by Jasper Snoek et al.](#)).<sup>1</sup>

2. Try Ensemble methods. Combining your best models will often produce better performance than running them individually.

3. Once you are confident about your final model, measure its performance on the test set to estimate the generalization error.

### WARNING

Don't tweak your model after measuring the generalization error: you would just start overfitting the test set.



## Present Your Solution

1. Document what you have done.
2. Create a nice presentation.
  - Make sure you highlight the big picture first.
3. Explain why your solution achieves the business objective.
4. Don't forget to present interesting points you noticed along the way.
  - Describe what worked and what did not.
  - List your assumptions and your system's limitations.
5. Ensure your key findings are communicated through beautiful visualizations or easy-to-remember statements (e.g., “the median income is the number-one predictor of housing prices”).

## Launch!

1. Get your solution ready for production (plug into production data inputs, write unit tests, etc.).
2. Write monitoring code to check your system's live performance at regular intervals and trigger alerts when it drops.
  - Beware of slow degradation: models tend to “rot” as data evolves.
  - Measuring performance may require a human pipeline (e.g., via a crowdsourcing service).
  - Also monitor your inputs' quality (e.g., a malfunctioning sensor sending random values, or another team's output becoming stale). This is particularly important for online learning systems.
3. Retrain your models on a regular basis on fresh data (automate as much as possible).

- 
- 1 Jasper Snoek et al., “Practical Bayesian Optimization of Machine Learning Algorithms,” *Proceedings of the 25th International Conference on Neural Information Processing Systems 2* (2012): 2951–2959.

[OceanofPDF.com](http://OceanofPDF.com)

# Appendix B. Autodiff

---

This appendix explains how TensorFlow's autodifferentiation (autodiff) feature works, and how it compares to other solutions.

Suppose you define a function  $f(x, y) = x^2y + y + 2$ , and you need its partial derivatives  $\partial f/\partial x$  and  $\partial f/\partial y$ , typically to perform Gradient Descent (or some other optimization algorithm). Your main options are manual differentiation, finite difference approximation, forward-mode autodiff, and reverse-mode autodiff. TensorFlow implements reverse-mode autodiff, but to understand it, it's useful to look at the other options first. So let's go through each of them, starting with manual differentiation.

## Manual Differentiation

The first approach to compute derivatives is to pick up a pencil and a piece of paper and use your calculus knowledge to derive the appropriate equation. For the function  $f(x, y)$  just defined, it is not too hard; you just need to use five rules:

- The derivative of a constant is 0.
- The derivative of  $\lambda x$  is  $\lambda$  (where  $\lambda$  is a constant).
- The derivative of  $x^\lambda$  is  $\lambda x^{\lambda-1}$ , so the derivative of  $x^2$  is  $2x$ .
- The derivative of a sum of functions is the sum of these functions' derivatives.
- The derivative of  $\lambda$  times a function is  $\lambda$  times its derivative.

From these rules, you can derive [Equation B-1](#).

Equation B-1. Partial derivatives of  $f(x, y)$

$$\begin{aligned}\frac{\partial f}{\partial x} &= \frac{\partial(x^2y)}{\partial x} + \frac{\partial y}{\partial x} + \frac{\partial 2}{\partial x} = y \frac{\partial(x^2)}{\partial x} + 0 + 0 = 2xy \\ \frac{\partial f}{\partial y} &= \frac{\partial(x^2y)}{\partial y} + \frac{\partial y}{\partial y} + \frac{\partial 2}{\partial y} = x^2 + 1 + 0 = x^2 + 1\end{aligned}$$

This approach can become very tedious for more complex functions, and you run the risk of making mistakes. Fortunately, there are other options. Let's look at finite difference approximation now.

## Finite Difference Approximation

Recall that the derivative  $h'(x_0)$  of a function  $h(x)$  at a point  $x_0$  is the slope of the function at that point. More precisely, the derivative is defined as the limit of the slope of a straight line going through this point  $x_0$  and another point  $x$  on the function, as  $x$  gets infinitely close to  $x_0$  (see [Equation B-2](#)).

Equation B-2. Definition of the derivative of a function  $h(x)$  at point  $x_0$

$$\begin{aligned}h'(x_0) &= \lim_{x \rightarrow x_0} \frac{h(x) - h(x_0)}{x - x_0} \\ &= \lim_{\varepsilon \rightarrow 0} \frac{h(x_0 + \varepsilon) - h(x_0)}{\varepsilon}\end{aligned}$$

So, if we wanted to calculate the partial derivative of  $f(x, y)$  with regard to  $x$  at  $x = 3$  and  $y = 4$ , we could compute  $f(3 + \varepsilon, 4) - f(3, 4)$  and divide the result by  $\varepsilon$ , using a very small value for  $\varepsilon$ . This type of numerical approximation of the derivative is called a *finite difference approximation*, and this specific equation is called *Newton's difference quotient*. That's exactly what the following code does:

```
def f(x, y):  
    return x**2*y + y + 2
```

```
def derivative(f, x, y, x_eps, y_eps):
    return (f(x + x_eps, y + y_eps) - f(x, y)) / (x_eps + y_eps)

df_dx = derivative(f, 3, 4, 0.00001, 0)
df_dy = derivative(f, 3, 4, 0, 0.00001)
```

Unfortunately, the result is imprecise (and it gets worse for more complicated functions). The correct results are respectively 24 and 10, but instead we get:

```
>>> df_dx
24.000039999805264
>>> df_dy
10.000000000331966
```

Notice that to compute both partial derivatives, we have to call  $f()$  at least three times (we called it four times in the preceding code, but it could be optimized). If there were 1,000 parameters, we would need to call  $f()$  at least 1,001 times. When you are dealing with large neural networks, this makes finite difference approximation way too inefficient.

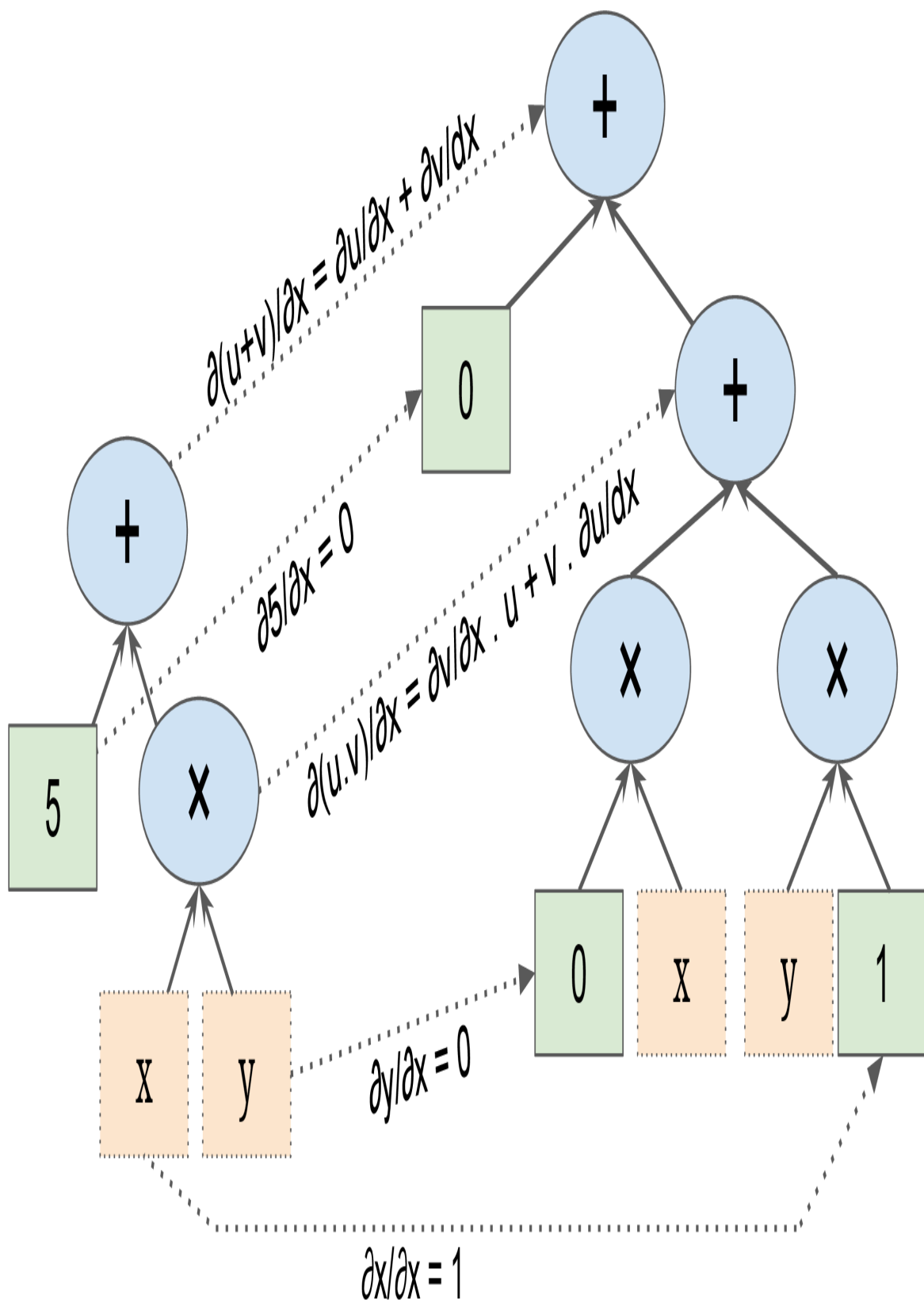
However, this method is so simple to implement that it is a great tool to check that the other methods are implemented correctly. For example, if it disagrees with your manually derived function, then your function probably contains a mistake.

So far, we have considered two ways to compute gradients: using manual differentiation and using finite difference approximation. Unfortunately, both were fatally flawed to train a large-scale neural network. So let's turn to autodiff, starting with forward mode.

## Forward-Mode Autodiff

Figure B-1 shows how forward-mode autodiff works on an even simpler function,  $g(x, y) = 5 + xy$ . The graph for that function is represented on the left. After forward-mode autodiff, we get the graph on the right, which

represents the partial derivative  $\partial g / \partial x = 0 + (0 \times x + y \times 1) = y$  (we could similarly obtain the partial derivative with regard to  $y$ ).



$$g(x, y) = 5 + xy$$

$$\frac{\partial x}{\partial x} = 1$$

$$\frac{\partial g}{\partial x} = 0 + (0 \cdot x + y \cdot 1) = y$$

Figure B-1. Forward-mode autodiff

The algorithm will go through the computation graph from the inputs to the outputs (hence the name “forward mode”). It starts by getting the partial derivatives of the leaf nodes. The constant node (5) returns the constant 0, since the derivative of a constant is always 0. The variable  $x$  returns the constant 1 since  $\partial x / \partial x = 1$ , and the variable  $y$  returns the constant 0 since  $\partial y / \partial x = 0$  (if we were looking for the partial derivative with regard to  $y$ , it would be the reverse).

Now we have all we need to move up the graph to the multiplication node in function  $g$ . Calculus tells us that the derivative of the product of two functions  $u$  and  $v$  is  $\partial(u \times v) / \partial x = \partial v / \partial x \times u + v \times \partial u / \partial x$ . We can therefore construct a large part of the graph on the right, representing  $0 \times x + y \times 1$ .

Finally, we can go up to the addition node in function  $g$ . As mentioned, the derivative of a sum of functions is the sum of these functions’ derivatives. So we just need to create an addition node and connect it to the parts of the graph we have already computed. We get the correct partial derivative:  $\partial g / \partial x = 0 + (0 \times x + y \times 1)$ .

However, this equation can be simplified (a lot). A few pruning steps can be applied to the computation graph to get rid of all unnecessary operations, and we get a much smaller graph with just one node:  $\partial g / \partial x = y$ . In this case simplification is fairly easy, but for a more complex function forward-mode autodiff can produce a huge graph that may be tough to simplify and lead to suboptimal performance.

Note that we started with a computation graph, and forward-mode autodiff produced another computation graph. This is called *symbolic differentiation*, and it has two nice features: first, once the computation graph of the derivative has been produced, we can use it as many times as we want to compute the derivatives of the given function for any value of  $x$  and  $y$ ; second, we can run forward-mode autodiff again on the resulting graph to get second-order derivatives if we ever need to (i.e., derivatives of derivatives). We could even compute third-order derivatives, and so on.



But it is also possible to run forward-mode autodiff without constructing a graph (i.e., numerically, not symbolically), just by computing intermediate results on the fly. One way to do this is to use *dual numbers*, which are weird but fascinating numbers of the form  $a + b\varepsilon$ , where  $a$  and  $b$  are real numbers and  $\varepsilon$  is an infinitesimal number such that  $\varepsilon^2 = 0$  (but  $\varepsilon \neq 0$ ). You can think of the dual number  $42 + 24\varepsilon$  as something akin to 42.0000...000024 with an infinite number of 0s (but of course this is simplified just to give you some idea of what dual numbers are). A dual number is represented in memory as a pair of floats. For example,  $42 + 24\varepsilon$  is represented by the pair (42.0, 24.0).

Dual numbers can be added, multiplied, and so on, as shown in **Equation B-3**.

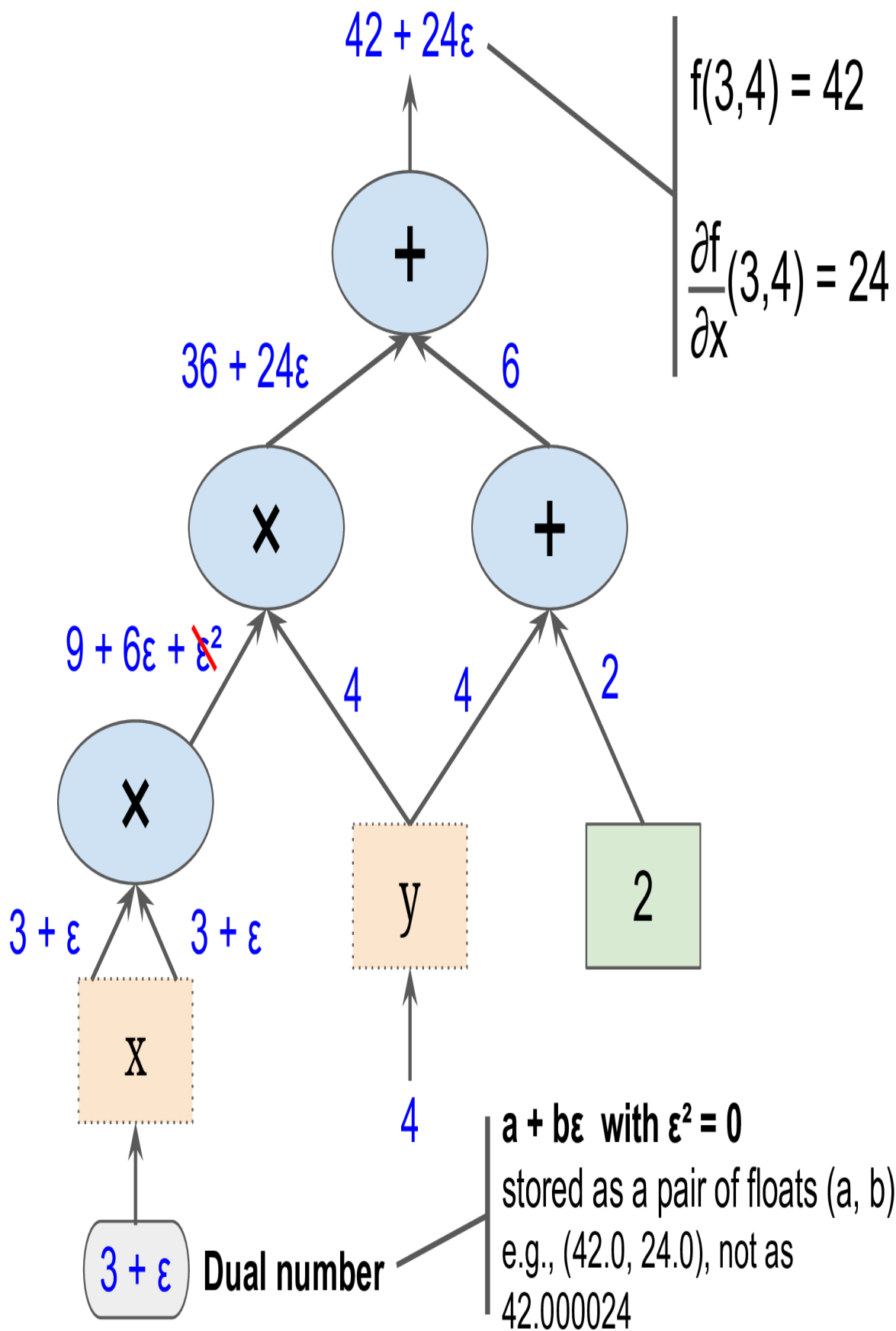
*Equation B-3. A few operations with dual numbers*

$$\lambda(a + b\varepsilon) = \lambda a + \lambda b\varepsilon$$

$$(a + b\varepsilon) + (c + d\varepsilon) = (a + c) + (b + d)\varepsilon$$

$$(a + b\varepsilon) \times (c + d\varepsilon) = ac + (ad + bc)\varepsilon + (bd)\varepsilon^2 = ac + (ad + bc)\varepsilon$$

Most importantly, it can be shown that  $h(a + b\varepsilon) = h(a) + b \times h'(a)\varepsilon$ , so computing  $h(a + \varepsilon)$  gives you both  $h(a)$  and the derivative  $h'(a)$  in just one shot. **Figure B-2** shows that the partial derivative of  $f(x, y)$  with regard to  $x$  at  $x = 3$  and  $y = 4$  (which I will write  $\partial f / \partial x (3, 4)$ ) can be computed using dual numbers. All we need to do is compute  $f(3 + \varepsilon, 4)$ ; this will output a dual number whose first component is equal to  $f(3, 4)$  and whose second component is equal to  $\partial f / \partial x (3, 4)$ .



*Figure B-2. Forward-mode autodiff using dual numbers*

To compute  $\partial f / \partial y(3, 4)$  we would have to go through the graph again, but this time with  $x = 3$  and  $y = 4 + \epsilon$ .

So forward-mode autodiff is much more accurate than finite difference approximation, but it suffers from the same major flaw, at least when there are many inputs and few outputs (as is the case when dealing with neural networks): if there were 1,000 parameters, it would require 1,000 passes through the graph to compute all the partial derivatives. This is where reverse-mode autodiff shines: it can compute all of them in just two passes through the graph. Let's see how.

## Reverse-Mode Autodiff

Reverse-mode autodiff is the solution implemented by TensorFlow. It first goes through the graph in the forward direction (i.e., from the inputs to the output) to compute the value of each node. Then it does a second pass, this time in the reverse direction (i.e., from the output to the inputs), to compute all the partial derivatives. The name “reverse mode” comes from this second pass through the graph, where gradients flow in the reverse direction. **Figure B-3** represents the second pass. During the first pass, all the node values were computed, starting from  $x = 3$  and  $y = 4$ . You can see those values at the bottom right of each node (e.g.,  $x \times x = 9$ ). The nodes are labeled  $n_1$  to  $n_7$  for clarity. The output node is  $n_7$ :  $f(3, 4) = n_7 = 42$ .

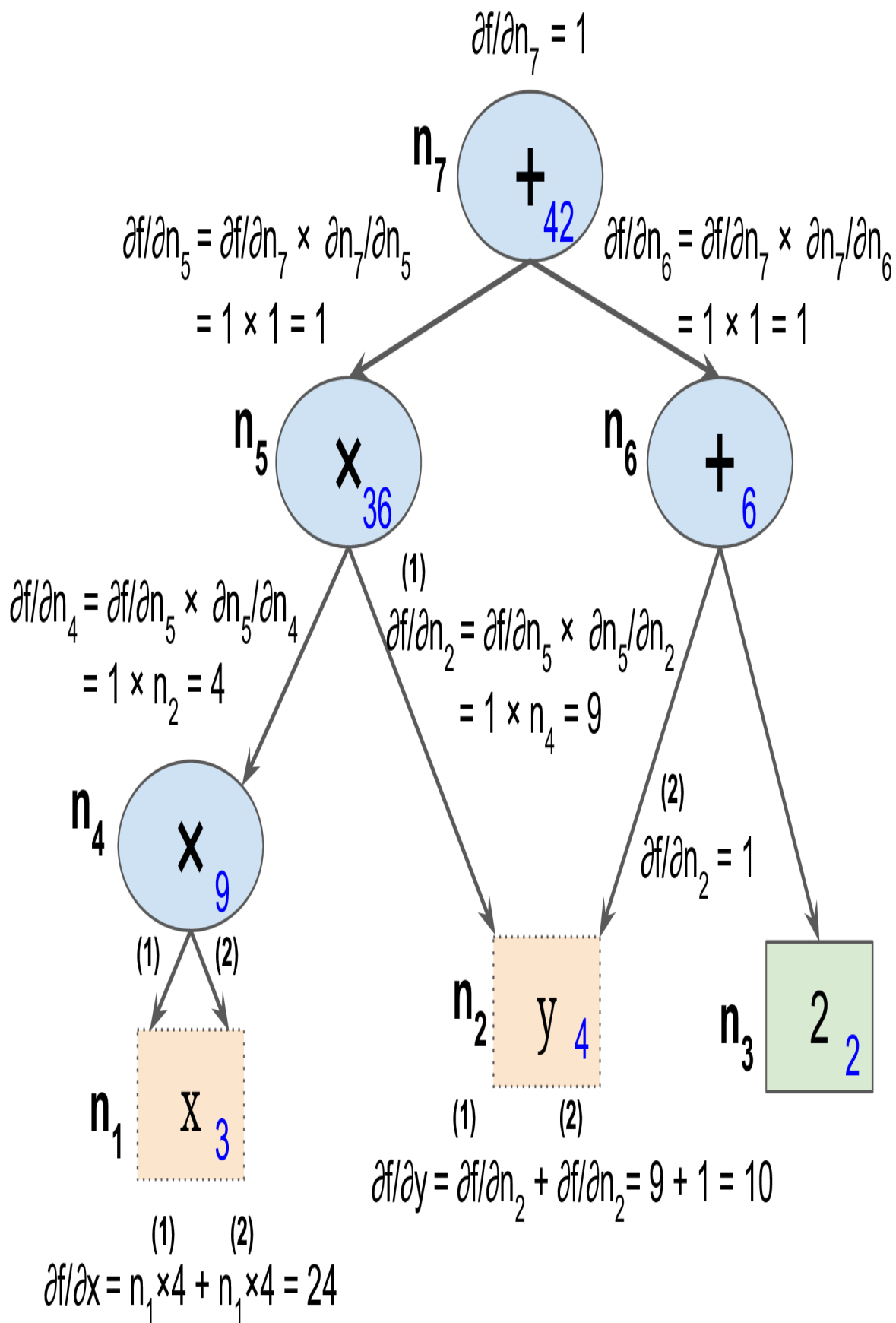


Figure B-3. Reverse-mode autodiff

The idea is to gradually go down the graph, computing the partial derivative of  $f(x, y)$  with regard to each consecutive node, until we reach the variable nodes. For this, reverse-mode autodiff relies heavily on the *chain rule*, shown in [Equation B-4](#).

Equation B-4. Chain rule

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_i} \times \frac{\partial n_i}{\partial x}$$

Since  $n_7$  is the output node,  $f = n_7$  so  $\partial f / \partial n_7 = 1$ .

Let's continue down the graph to  $n_5$ : how much does  $f$  vary when  $n_5$  varies? The answer is  $\partial f / \partial n_5 = \partial f / \partial n_7 \times \partial n_7 / \partial n_5$ . We already know that  $\partial f / \partial n_7 = 1$ , so all we need is  $\partial n_7 / \partial n_5$ . Since  $n_7$  simply performs the sum  $n_5 + n_6$ , we find that  $\partial n_7 / \partial n_5 = 1$ , so  $\partial f / \partial n_5 = 1 \times 1 = 1$ .

Now we can proceed to node  $n_4$ : how much does  $f$  vary when  $n_4$  varies? The answer is  $\partial f / \partial n_4 = \partial f / \partial n_5 \times \partial n_5 / \partial n_4$ . Since  $n_5 = n_4 \times n_2$ , we find that  $\partial n_5 / \partial n_4 = n_2$ , so  $\partial f / \partial n_4 = 1 \times n_2 = 4$ .

The process continues until we reach the bottom of the graph. At that point we will have calculated all the partial derivatives of  $f(x, y)$  at the point  $x = 3$  and  $y = 4$ . In this example, we find  $\partial f / \partial x = 24$  and  $\partial f / \partial y = 10$ . Sounds about right!

Reverse-mode autodiff is a very powerful and accurate technique, especially when there are many inputs and few outputs, since it requires only one forward pass plus one reverse pass per output to compute all the partial derivatives for all outputs with regard to all the inputs. When training neural networks, we generally want to minimize the loss, so there is a single output (the loss), and hence only two passes through the graph are needed to compute the gradients. Reverse-mode autodiff can also handle functions that are not entirely differentiable, as long as you ask it to compute the partial derivatives at points that are differentiable.

In **Figure B-3**, the numerical results are computed on the fly, at each node. However, that's not exactly what TensorFlow does: instead, it creates a new computation graph. In other words, it implements *symbolic* reverse-mode autodiff. This way, the computation graph to compute the gradients of the loss with regard to all the parameters in the neural network only needs to be generated once, and then it can be executed over and over again, whenever the optimizer needs to compute the gradients. Moreover, this makes it possible to compute higher-order derivatives if needed.

### TIP

If you ever want to implement a new type of low-level TensorFlow operation in C++, and you want to make it compatible with autodiff, then you will need to provide a function that returns the partial derivatives of the function's outputs with regard to its inputs. For example, suppose you implement a function that computes the square of its input:  $f(x) = x^2$ . In that case you would need to provide the corresponding derivative function:  $f'(x) = 2x$ .

# Appendix C. Special Data Structures

---

In this appendix we will take a very quick look at the data structures supported by TensorFlow, beyond regular float or integer tensors. This includes strings, ragged tensors, sparse tensors, tensor arrays, sets, and queues.

## Strings

Tensors can hold byte strings, which is useful in particular for natural language processing (see [Chapter 16](#)):

```
>>> tf.constant(b"hello world")
<tf.Tensor: shape=(), dtype=string, numpy=b'hello world'>
```

If you try to build a tensor with a Unicode string, TensorFlow automatically encodes it to UTF-8:

```
>>> tf.constant("café")
<tf.Tensor: shape=(), dtype=string, numpy=b'caf\xc3\xa9'>
```

It is also possible to create tensors representing Unicode strings. Just create an array of 32-bit integers, each representing a single Unicode code point:<sup>1</sup>

```
>>> u = tf.constant([ord(c) for c in "café"])
>>> u
<tf.Tensor: shape=(4,), [...], numpy=array([ 99,  97, 102, 233],
dtype=int32)>
```

## NOTE

In tensors of type `tf.string`, the string length is not part of the tensor's shape. In other words, strings are considered as atomic values. However, in a Unicode string tensor (i.e., an `int32` tensor), the length of the string is part of the tensor's shape.

The `tf.strings` package contains several functions to manipulate string tensors, such as `length()` to count the number of bytes in a byte string (or the number of code points if you set `unit="UTF8_CHAR"`), `unicode_encode()` to convert a Unicode string tensor (i.e., `int32` tensor) to a byte string tensor, and `unicode_decode()` to do the reverse:

```
>>> b = tf.strings.unicode_encode(u, "UTF-8")
>>> b
<tf.Tensor: shape=(), dtype=string, numpy=b'caf\xc3\xa9'>
>>> tf.strings.length(b, unit="UTF8_CHAR")
<tf.Tensor: shape=(), dtype=int32, numpy=4>
>>> tf.strings.unicode_decode(b, "UTF-8")
<tf.Tensor: shape=(4,), [...], numpy=array([ 99,  97, 102, 233],
dtype=int32)>
```

You can also manipulate tensors containing multiple strings:

```
>>> p = tf.constant(["Café", "Coffee", "caffè", " "])
>>> tf.strings.length(p, unit="UTF8_CHAR")
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([4, 6, 5, 2],
dtype=int32)>
>>> r = tf.strings.unicode_decode(p, "UTF8")
>>> r
<tf.RaggedTensor [[67, 97, 102, 233], [67, 111, 102, 102, 101,
101], [99, 97,
102, 102, 232], [21654, 21857]]>
```

Notice that the decoded strings are stored in a `RaggedTensor`. What is that?



## Ragged Tensors

A ragged tensor is a special kind of tensor that represents a list of arrays of different sizes. More generally, it is a tensor with one or more *ragged dimensions*, meaning dimensions whose slices may have different lengths. In the ragged tensor `r`, the second dimension is a ragged dimension. In all ragged tensors, the first dimension is always a regular dimension (also called a *uniform dimension*).

All the elements of the ragged tensor `r` are regular tensors. For example, let's look at the second element of the ragged tensor:

```
>>> r[1]
<tf.Tensor: [...], numpy=array([ 67, 111, 102, 102, 101, 101],
dtype=int32)>
```

The `tf.ragged` package contains several functions to create and manipulate ragged tensors. Let's create a second ragged tensor using `tf.ragged.constant()` and concatenate it with the first ragged tensor, along axis 0:

```
>>> r2 = tf.ragged.constant([[65, 66], [], [67]])
>>> tf.concat([r, r2], axis=0)
<tf.RaggedTensor [[67, 97, 102, 233], [67, 111, 102, 102, 101,
101], [99, 97,
102, 102, 232], [21654, 21857], [65, 66], [], [67]]>
```

The result is not too surprising: the tensors in `r2` were appended after the tensors in `r` along axis 0. But what if we concatenate `r` and another ragged tensor along axis 1?

```
>>> r3 = tf.ragged.constant([[68, 69, 70], [71], [], [72, 73]])
>>> print(tf.concat([r, r3], axis=1))
<tf.RaggedTensor [[67, 97, 102, 233, 68, 69, 70], [67, 111, 102,
102, 101, 101,
71], [99, 97, 102, 102, 232], [21654, 21857, 72, 73]]>
```

This time, notice that the  $i^{\text{th}}$  tensor in `r` and the  $i^{\text{th}}$  tensor in `r3` were concatenated. Now that's more unusual, since all of these tensors can have different lengths.

If you call the `to_tensor()` method, the ragged tensor gets converted to a regular tensor, padding shorter tensors with zeros to get tensors of equal lengths (you can change the default value by setting the `default_value` argument):

```
>>> r.to_tensor()
<tf.Tensor: shape=(4, 6), dtype=int32, numpy=
array([[ 67,   97,  102,  233,    0,    0],
       [ 67,  111,  102,  102,  101,  101],
       [ 99,   97,  102,  102,  232,    0],
       [21654, 21857,    0,    0,    0,    0]], dtype=int32)>
```

Many TF operations support ragged tensors. For the full list, see the documentation of the `tf.RaggedTensor` class.

## Sparse Tensors

TensorFlow can also efficiently represent *sparse tensors* (i.e., tensors containing mostly zeros). Just create a `tf.SparseTensor`, specifying the indices and values of the nonzero elements and the tensor's shape. The indices must be listed in “reading order” (from left to right, and top to bottom). If you are unsure, just use `tf.sparse.reorder()`. You can convert a sparse tensor to a dense tensor (i.e., a regular tensor) using `tf.sparse.to_dense()`:

```
>>> s = tf.SparseTensor(indices=[[0, 1], [1, 0], [2, 3]],
...                     values=[1., 2., 3.],
...                     dense_shape=[3, 4])
...
>>> tf.sparse.to_dense(s)
<tf.Tensor: shape=(3, 4), dtype=float32, numpy=
array([[0., 1., 0., 0.],
       [2., 0., 0., 0.],
       [0., 0., 0., 3.]], dtype=float32)>
```

Note that sparse tensors do not support as many operations as dense tensors. For example, you can multiply a sparse tensor by any scalar value, and you get a new sparse tensor, but you cannot add a scalar value to a sparse tensor, as this would not return a sparse tensor:

```
>>> s * 42.0
<tensorflow.python.framework.sparse_tensor.SparseTensor at
0x7f84a6749f10>
>>> s + 42.0
[...] TypeError: unsupported operand type(s) for +:
'SparseTensor' and 'float'
```

## Tensor Arrays

A `tf.TensorArray` represents a list of tensors. This can be handy in dynamic models containing loops, to accumulate results and later compute some statistics. You can read or write tensors at any location in the array:

```
array = tf.TensorArray(dtype=tf.float32, size=3)
array = array.write(0, tf.constant([1., 2.]))
array = array.write(1, tf.constant([3., 10.]))
array = array.write(2, tf.constant([5., 7.]))
tensor1 = array.read(1) # => returns (and zeros out!)
tf.constant([3., 10.])
```

By default, reading an item also replaces it with a tensor of the same shape but full of zeros. You can set `clear_after_read` to `False` if you don't want this.

### WARNING

When you write to the array, you must assign the output back to the array, as shown in this code example. If you don't, although your code will work fine in eager mode, it will break in graph mode (these modes are discussed in [Chapter 12](#)).

By default, a `TensorArray` has a fixed size which is set upon creation. Alternatively, you can set `size=0` and `dynamic_size=True` to let the

array grow automatically when needed. However, this will hinder performance, so if you know the `size` in advance, it's better to use a fixed size array. You must also specify the `dtype`, and all elements must have the same shape as the first one written to the array.

You can stack all the items into a regular tensor by calling the `stack()` method:

```
>>> array.stack()
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[1., 2.],
       [0., 0.],
       [5., 7.]], dtype=float32)>
```

## Sets

TensorFlow supports sets of integers or strings (but not floats). It represents sets using regular tensors. For example, the set  $\{1, 5, 9\}$  is just represented as the tensor  $\begin{bmatrix} 1 & 5 & 9 \end{bmatrix}$ . Note that the tensor must have at least two dimensions, and the sets must be in the last dimension. For example,  $\begin{bmatrix} 1 & 5 & 9 \\ 2 & 5 & 11 \end{bmatrix}$  is a tensor holding two independent sets:  $\{1, 5, 9\}$  and  $\{2, 5, 11\}$ .

The `tf.sets` package contains several functions to manipulate sets. For example, let's create two sets and compute their union (the result is a sparse tensor, so we call `to_dense()` to display it):

```
>>> a = tf.constant([[1, 5, 9]])
>>> b = tf.constant([[5, 6, 9, 11]])
>>> u = tf.sets.union(a, b)
>>> u
<tensorflow.python.framework.sparse_tensor.SparseTensor at
0x132b60d30>
>>> tf.sparse.to_dense(u)
<tf.Tensor: [...], numpy=array([[ 1,  5,  6,  9, 11]],
dtype=int32)>
```

You can also compute the union of multiple pairs of sets simultaneously. If some sets are shorter than others, you must pad them with a padding value, such as 0:

```
>>> a = tf.constant([[1, 5, 9], [10, 0, 0]])
>>> b = tf.constant([[5, 6, 9, 11], [13, 0, 0, 0]])
>>> u = tf.sets.union(a, b)
>>> tf.sparse.to_dense(u)
<tf.Tensor: [...] numpy=array([[ 1,  5,  6,  9, 11],
                                [ 0, 10, 13,  0,  0]],
                                dtype=int32)>
```

If you prefer to use a different padding value, such as  $-1$ , then you must set `default_value=-1` when calling `to_dense()`.

### WARNING

The default `default_value` is 0, so when dealing with string sets, you must set the `default_value` (e.g., to an empty string).

Other functions available in `tf.sets` include `difference()`, `intersection()`, and `size()`, which are self-explanatory. If you want to check whether or not a set contains some given values, you can compute the intersection of that set and the values. If you want to add some values to a set, you can compute the union of the set and the values.

## Queues

A queue is a data structure to which you can push data records, and later pull them out. TensorFlow implements several types of queues in the `tf.queue` package. They used to be very important when implementing efficient data loading and preprocessing pipelines, but the `tf.data` API has essentially rendered them useless (except perhaps in some rare cases) because it is much simpler to use and provides all the tools you need to

build efficient pipelines. For the sake of completeness, though, let's take a quick look at them.

The simplest kind of queue is the first-in, first-out (FIFO) queue. To build it, you need to specify the maximum number of records it can contain. Moreover, each record is a tuple of tensors, so you must specify the type of each tensor, and optionally their shapes. For example, the following code example creates a FIFO queue with maximum three records, each containing a tuple with a 32-bit integer and a string. Then it pushes two records to it, looks at the size (which is 2 at this point), and pulls a record out:

```
>>> q = tf.queue.FIFOQueue(3, [tf.int32, tf.string], shapes=[(),
    ()])
>>> q.enqueue([10, b"windy"])
>>> q.enqueue([15, b"sunny"])
>>> q.size()
<tf.Tensor: shape=(), dtype=int32, numpy=2>
>>> q.dequeue()
[<tf.Tensor: shape=(), dtype=int32, numpy=10>,
 <tf.Tensor: shape=(), dtype=string, numpy=b'windy'>]
```

It is also possible to enqueue and dequeue multiple records at once using `enqueue_many()` and `dequeue_many()` (to use `dequeue_many()`, you must specify the `shapes` argument when you create the queue, as we did above):

```
>>> q.enqueue_many([[13, 16], [b'cloudy', b'rainy']])
>>> q.dequeue_many(3)
[<tf.Tensor: [...], numpy=array([15, 13, 16], dtype=int32)>,
 <tf.Tensor: [...], numpy=array([b'sunny', b'cloudy', b'rainy'],
 dtype=object)>]
```

Other queue types include:

### *PaddingFIFOQueue*

Same as `FIFOQueue`, but its `dequeue_many()` method supports dequeuing multiple records of different shapes. It automatically pads

the shortest records to ensure all the records in the batch have the same shape.

### *PriorityQueue*

A queue that dequeues records in a prioritized order. The priority must be a 64-bit integer included as the first element of each record.

Surprisingly, records with a lower priority will be dequeued first.

Records with the same priority will be dequeued in FIFO order.

### *RandomShuffleQueue*

A queue whose records are dequeued in random order. This was useful to implement a shuffle buffer before `tf.data` existed.

If a queue is already full and you try to enqueue another record, the `enqueue*()` method will freeze until a record is dequeued by another thread. Similarly, if a queue is empty and you try to dequeue a record, the `dequeue*()` method will freeze until records are pushed to the queue by another thread.

---

<sup>1</sup> If you are not familiar with Unicode code points, please check out <https://homl.info/unicode>.

# Appendix D. TensorFlow Graphs

---

In this appendix, we will explore the graphs generated by TF Functions (see [Chapter 12](#)).

## TF Functions and Concrete Functions

TF Functions are polymorphic, meaning they support inputs of different types (and shapes). For example, consider the following `tf_cube()` function:

```
@tf.function
def tf_cube(x):
    return x ** 3
```

Every time you call a TF Function with a new combination of input types or shapes, it generates a new *concrete function*, with its own graph specialized for this particular combination. Such a combination of argument types and shapes is called an *input signature*. If you call the TF Function with an input signature it has already seen before, it will reuse the concrete function it generated earlier. For example, if you call `tf_cube(tf.constant(3.0))`, the TF Function will reuse the same concrete function it used for `tf_cube(tf.constant(2.0))` (for float32 scalar tensors). But it will generate a new concrete function if you call `tf_cube(tf.constant([2.0]))` or `tf_cube(tf.constant([3.0]))` (for float32 tensors of shape [1]), and yet another for `tf_cube(tf.constant([[1.0, 2.0], [3.0, 4.0]]))` (for float32 tensors of shape [2, 2]). You can get the concrete function for a particular combination of inputs by calling the TF Function's `get_concrete_function()` method. It can then be called



like a regular function, but it will only support one input signature (in this example, float32 scalar tensors):

```
>>> concrete_function =  
tf_cube.get_concrete_function(tf.constant(2.0))  
>>> concrete_function  
<ConcreteFunction tf_cube(x) at 0x7F84411F4250>  
>>> concrete_function(tf.constant(2.0))  
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

Figure D-1 shows the `tf_cube()` TF Function, after we called `tf_cube(2)` and `tf_cube(tf.constant(2.0))`: two concrete functions were generated, one for each signature, each with its own optimized *function graph* (`FuncGraph`), and its own *function definition* (`FunctionDef`). A function definition points to the parts of the graph that correspond to the function's inputs and outputs. In each `FuncGraph`, the nodes (ovals) represent operations (e.g., power, constants, or placeholders for arguments like `x`), while the edges (the solid arrows between the operations) represent the tensors that will flow through the graph. The concrete function on the left is specialized for `x = 2`, so TensorFlow managed to simplify it to just output 8 all the time (note that the function definition does not even have an input). The concrete function on the right is specialized for float32 scalar tensors, and it could not be simplified. If we call `tf_cube(tf.constant(5.0))`, the second concrete function will be called, the placeholder operation for `x` will output 5.0, then the power operation will compute `5.0 ** 3`, so the output will be 125.0.

